

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	ARM 开发环境搭建及 LED 灯实验

一、 实验目的与要求：

（1）掌握嵌入式软件开发环境的建立过程；

（2）初步掌握嵌入式程序的编译和调试过程；

（3）熟悉 eclipse 开发工具建立汇编工程和仿真。

二、实验设备：

1. FS-JTAG 仿真器、USB 转串口线、电源
2. PC 机（OS：Windows 7）
3. Eclipse for ARM
4. GCC 编译工具
5. Yagarto 工具包
6. JRE
7. Exynos4412 实验箱

三、实验方法（原理、流程图）

原理：

仿真器简介：



图 FS-JTAG 仿真器

FS_JTAG ARM 仿真器是继华清远见研发中心自主研发的 Cortex 系列开发平台，及配套案例资料和技术支持获得业内合作企业及参训学员的一致好评之后，经过几个月的潜心研究和专注努力，由华清远见研发中心自主研发 Cortex-A 系列 ARM 仿真器。这无疑为业内合作企业、合作院校及广大培训学员带来了非常好的消息。

了解行业和相关技术的人都知道，功能完善的 ARM 仿真器和软件调试环境对于学习 ARM 处理器的工作原理和核心知识来说至关重要。由于之前多年的技术发展和行业实践，针对 Cortex-M 系列、ARM7、ARM9、以及 ARM11 系列处理器，市场上都已经有很多成熟的、价廉物美的仿真器可供选择。而对于目前最新流行的 ARM 应用处理器 Cortex-A 系列来说，业内的技术工程师们却很难找到价格合适，功能完善的仿真器，国外动辄几千甚至上万的价格，让很多人感叹之余，只能望而却步。

华清远见研发中心为了推进 Cortex-A 系列 ARM 处理器的教学，提高合作企业及合作院校广大技术爱好者和培训学员的学习效率，最新生产研发出 FS-JTAG 仿真器，该款仿真器可以仿真 Cortex-M3、ARM7、ARM9、ARM11、Cortex-A8、Cortex-A9 等多个 ARM 处理器

系列。同时，华清远见研发中心也在 FS-S5PC100、FS210、FS4412 教学平台上使用 FS-JTAG 环境开发了全套的裸机接口代码，再配合研发中心出版的图书及实验指导书，相信一定会成为业内最实用的 ARM 处理器配套教学平台和辅助学习资料。

流程图：

实验步骤一：开发环境搭建

```
    |
    |---> 1.  安装 FS-JTAG 相关工具
    |          |--->  安装 GCC 编译工具
    |          |--->  安装 Yagarto 工具包
    |          |--->  安装 FS-JTAG 调试软件
    |          |--->  安装 JRE
    |          |--->  安装 FS-JTAG 驱动
    |
    |---> 2.  连接硬件平台
    |          |--->  连接仿真器、USB 转串口线、电源
    |
    |---> 3. USB 转串口驱动安装
    |          |--->  运行 CH341SER.EXE，完成驱动安装
    |
    |---> 4. Putty 串口终端配置
               |--->  运行 PUTTY.EXE，配置串口终端连接
```

实验步骤二：导入一个新工程

```
    |---> 1.  导入汇编工程
    |          |--->  打开 Eclipse 开发工具，添加 LED 工程
    |---> 2.  编译程序
    |          |--->  点击编译按钮进行程序编译
    |---> 3.  调试工程
               |---> a.  配置 FS-JTAG 调试工具
               |          |--->  打开 FS-JTAG 软件，连接目标板
               |---> b.  配置调试工具
               |          |--->  配置 Main 选项
               |          |--->  配置 Debugger 选项
               |          |--->  配置 Command 选项
               |---> c.  进入调试
                          |--->  点击调试按钮进行调试
```

实验步骤三：编写汇编语言，实现简单加法实验

流程图见后续实验部分。

四、实验过程、步骤及内容

说明：部分实验图标注日期为 2014 年，为实验指导书中清晰示例图，仅为丰富实验指导书内容所置，不影响实验报告撰写。

步骤一：开发环境搭建

1. FS-JTAG 相关工具安装

（1） 安装 GCC 编译工具（此电脑已安装，无需进行）

（2） 安装 Yagarto 工具包（此电脑已安装，无需进行）

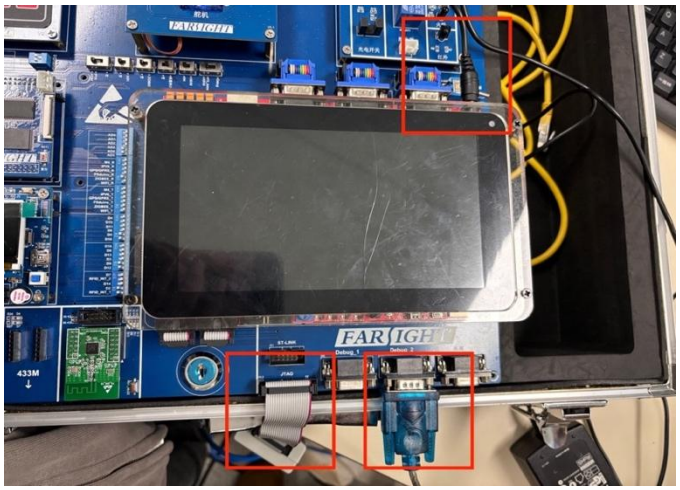
（3） 安装 FS-JTAG 调试软件（此电脑已安装，无需进行）

（4） 安装 JRE（此电脑已安装，无需进行）

（5） 安装 FS-JTAG 驱动（此电脑已安装，无需进行）

2. 连接硬件平台

按下图所示，连接仿真器、USB 转串口线、电源。



3. USB 转串口驱动安装

运行“华清远见-CORTEXA9 资料\工具软件\工具软件\Windows\USB 转串口驱动\CH340\CH341SER.EXE””

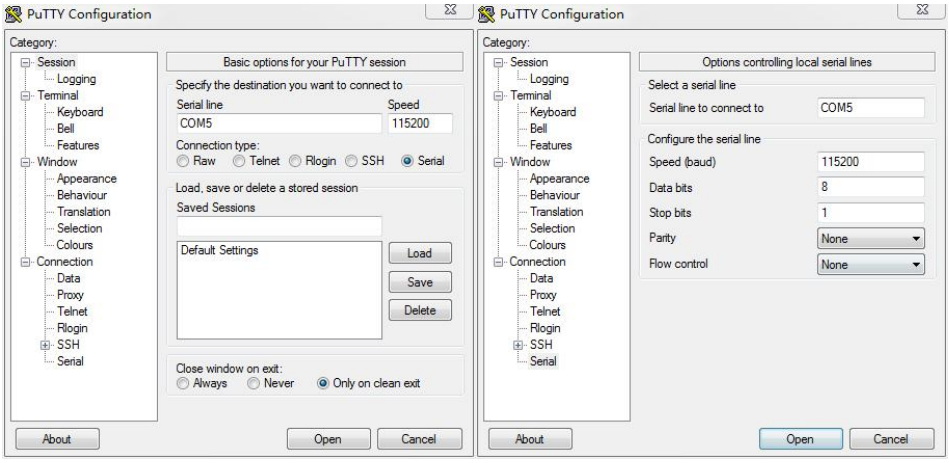


等待 20 秒左右，系统会提示安装完成。可以在设备管理器中查看到串口的信息，从而确定串口号。



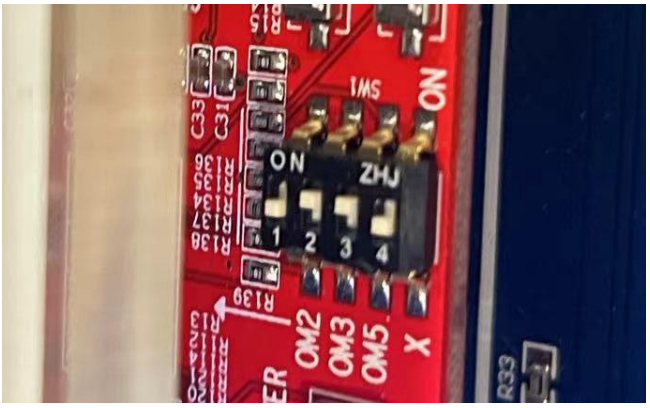
4. Putty 串口终端配置

运行“华清远见-CORTEXA9 资料\工具软件\Windows\串口调试工具\PUTTY.EXE”。

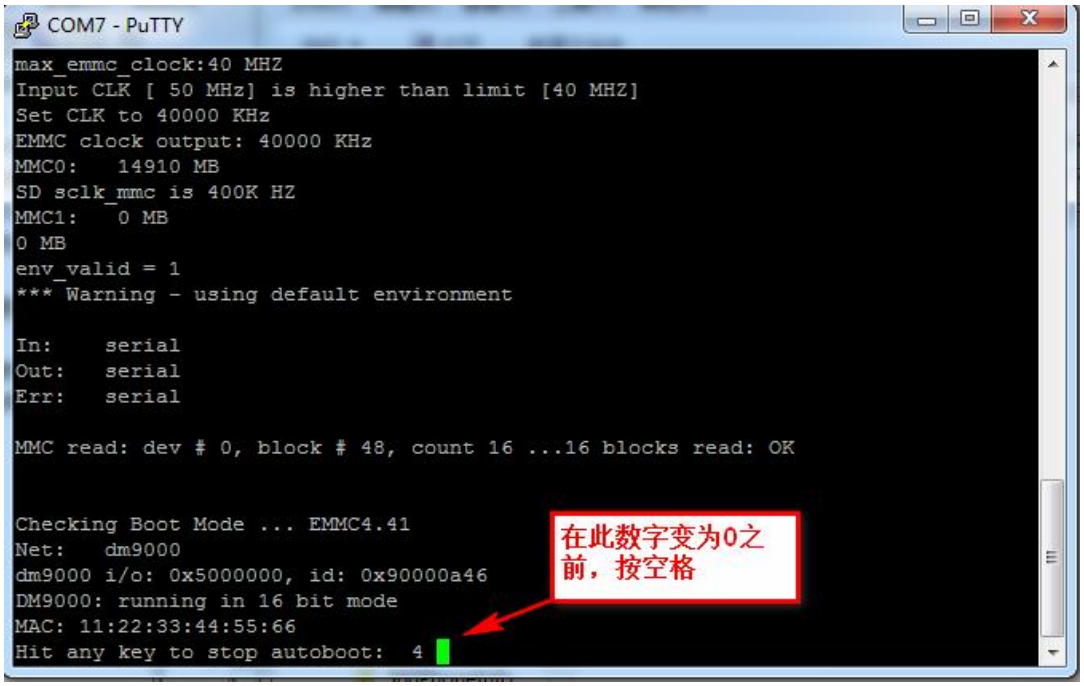


COM7 是串口号，不同机器、不同接口都有差异，请查看设备管理器中的信息。最后点击“Open”打开串口。

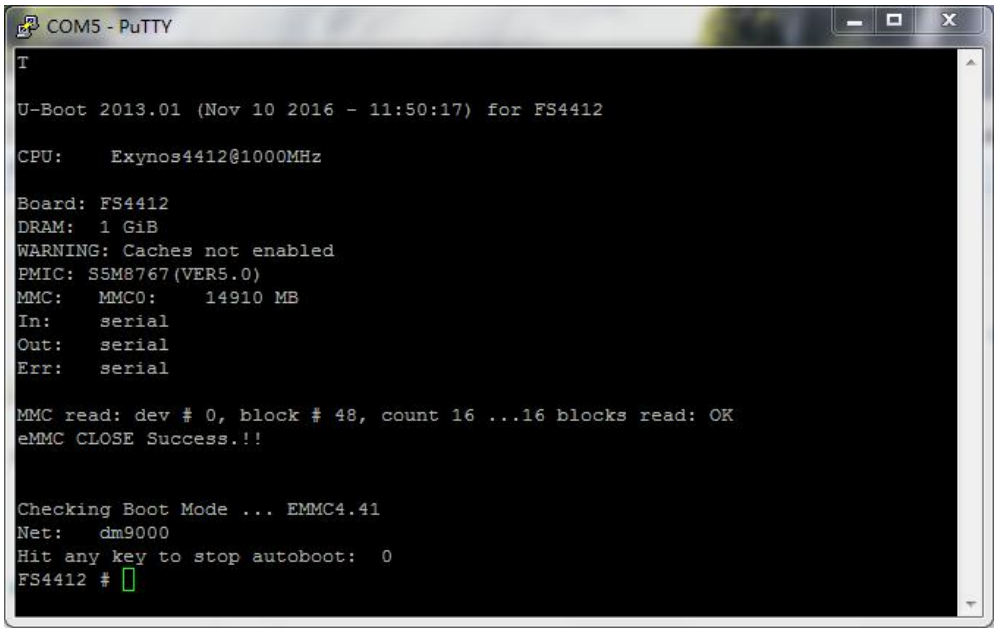
关闭开发板电源，将拨码开关 SW1 调至 0110(EMMC 启动模式)，然后打开电源。



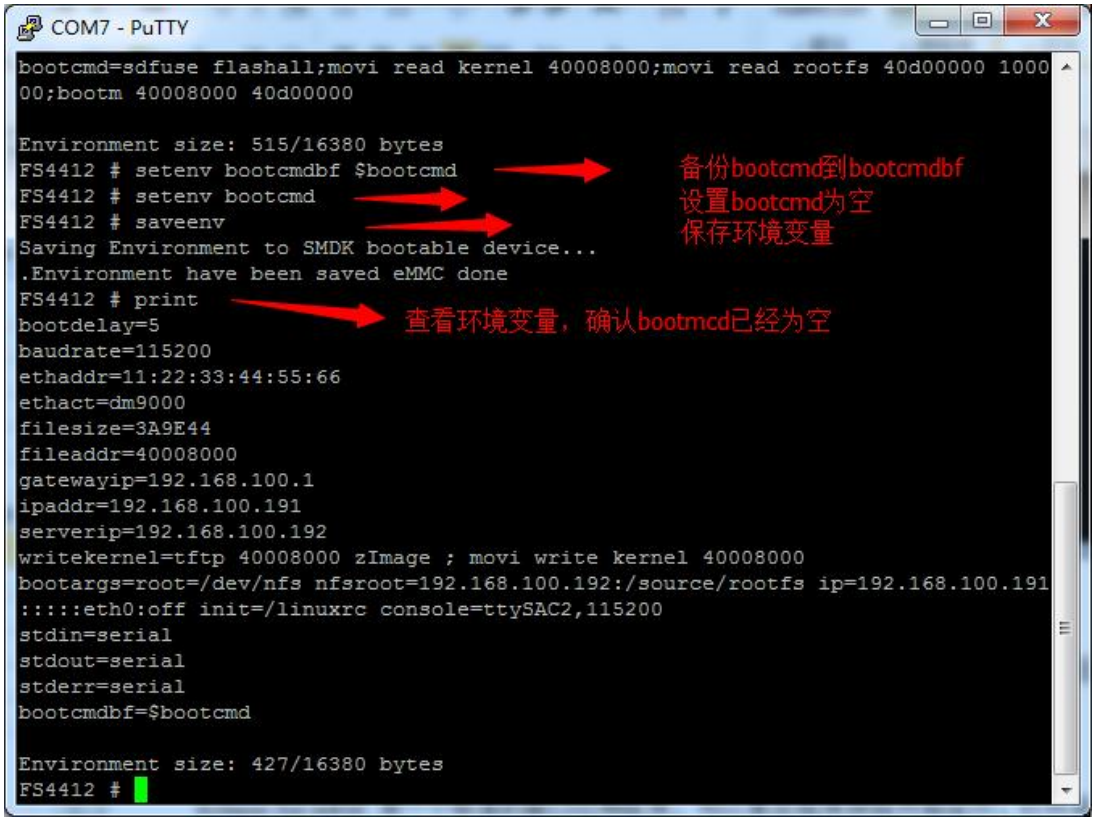
给开发板上电，此时串口终端会显示：



让系统停留在下图状态。



注意：以后每次连接仿真前，都需要确定处于此状态，保证不要启动到 Linux，因为启动到 Linux 后，MMU 功能会打开，导致仿真器无法正常使用。也可以改变 uboot 的 bootcmd 命令，让 uboot 不引导 linux。方法如下图所示：在 uboot 终端输入：



重新启动 uboot，自动停在 uboot 终端。

步骤二：创建（导入）一个新工程

1. 创建一个汇编工程

在一个汇编工程中,必须包含如下必要的文件：

```
├─工程名称
│      ├─led_assembly.S//存放汇编工程源代码
│      ├─Makefile                    //用来定义整个工程的编译规则
│      ├─map.lds//链接脚本文件
│      ├─Exynos4412.init//仿真初始化文件
```

进入主界面后，选择“File →New→C Project ”命令，Eclipse 将打开一个标准对话框， 输入希望新建工程的名字并单击“Finish ”按钮即可创建一个新的工程。

点击”Finish”，工程创建完成。

创建完成后，会在工程工作界面里有你创建的工程名，接下来添加工程必要文件。

（1） 新建一个 Makefile 文件

在创建一个新的工程后，选择“File →New→Other”命令，在弹出的对话框 New 中的 “General ”下单击 File ，然后单击 Next；然后选择所要指定的工程后，在文件名文本框中 输入文件名 Makefile ，单击“File”按钮。

点击“Next ”，弹出 Enter or select the parent tolde 对话框。

选择工程名，并在 File name 框内输入创建的文件名字

（2） 新建一个脚本文件

选择“File →New→Other ”命令，在弹出的对话框中的 General 下单击 file ，然后

单击 next；然后选择所要指定的工程后，在文件名文本框中输入文件名 Exynos4412.init ，单击 “Finish ”按钮。

（3） 新建一个链接脚本文件

选择“File →New→Other”命令，在弹出的对话框中的 General 下单击 file ，然后单击 next；然后选择所要指定的工程后，在文件名文本框中输入文件名 map.lds ，单击 “Finish ” 按钮。

（4） 新建一个汇编源文件

选择“File →New→Other”命令，在弹出的对话框中的 General 下单击 file ，然后单击 next；然后选择所要指定的工程后，在文件名文本框中输入文件名 led_assembly.S ，单击 “Finish ”按钮。

至此工程及所需文件已经基本创建完成，为了能够实现工程的相关功能，我们必须在 对应的文件中添加相应的代码。

（1） 在汇编源文件（led_assembly.S）中输入汇编代码

```
.equ   GPF3CON,   0x114001E0
.equ   GPF3DAT,   0x114001E4
.text
.global _start
_start:
        LDR         R0,=GPF3CON
        LDR     R1,=0X10000
        STR     R1,[R0]         @//写控制寄存器，IO 引脚使能为输出
LOOP:
        LDR         R0,=GPF3DAT
        MOV     R1,#0X10        @//点亮 led4
        STR     R1,[R0]
        LDR     R2,=0XFFFFFfF   @//延时
LOOP1:
        SUB         R2,R2,#1
        CMP     R2,#0
        BNE     LOOP1
        MOV     R1,#0X0         @//熄灭 led4
        STR     R1,[R0]
        LDR     R2,=0XFFFFFfF   @//延时
LOOP2:
        SUB         R2,R2,#1
        CMP     R2,#0
        BNE     LOOP2
        B       LOOP
        .end

```

（2） 在 map.lds 中输入如下信息：

```
OUTPUT_FORMAT("elf32-littlearm", "elf32-littlearm", "elf32-littlearm")
/*OUTPUT_FORMAT("elf32-arm", "elf32-arm", "elf32-arm")*/
OUTPUT_ARCH(arm)
ENTRY(_start)
SECTIONS
{
    . = 0x40008000;
    . = ALIGN(4);
    .text      :
    {
       led_assembly.o(.text)
        *(.text)
    }
    . = ALIGN(4);
    .rodata :
    { *(.rodata) }
    . = ALIGN(4);
    .data :
    { *(.data) }
    . = ALIGN(4);
    .bss :
     { *(.bss) }
}

```

（3） 编写 MakeFile 文件编译规则，在 MakeFile 中输入如下信息

```
SHELL=C:/Windows/system32/cmd.exe

all:clean
    arm-none-eabi-gcc-4.6.2   -O0   -g   -c   -o   led_assembly.o
led_assembly.S
    arm-none-eabi-ld                led_assembly.o  -Tmap.lds  -o
led_assembly.elf
    arm-none-eabi-objcopy       -O  binary  -S  led_assembly.elf
led_assembly.bin
    arm-none-eabi-objdump   -D led_assembly.elf > led_assembly.dis

clean:
    rm -rf *.o *.bin *.elf *.dis
```

注意：MakeFile 每条命令的开头必须以[Tab]键开头

（4） 在 Exynos4412.init 文件中输入如下信息：

```
target remote localhost:3333
monitor halt reset
```

所有文件编辑完成后，保存文件，创建好的工程如下图所示：

点击编译图标（或者快捷键“Ctrl + B ”）

至此 ARM 裸板汇编工程已经创建完成，如暂时不使用该工程时我们可以 Close project， 下次使用时再 Open project （同时只能有一个工程是打开的），该工程相关配置不变。

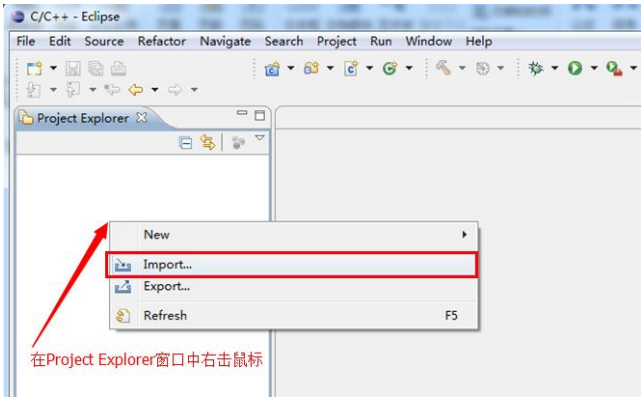
2. 导入一个汇编工程

打开 Eclipse 开发工具，在 Project Explorer 中添加 LED 工程。

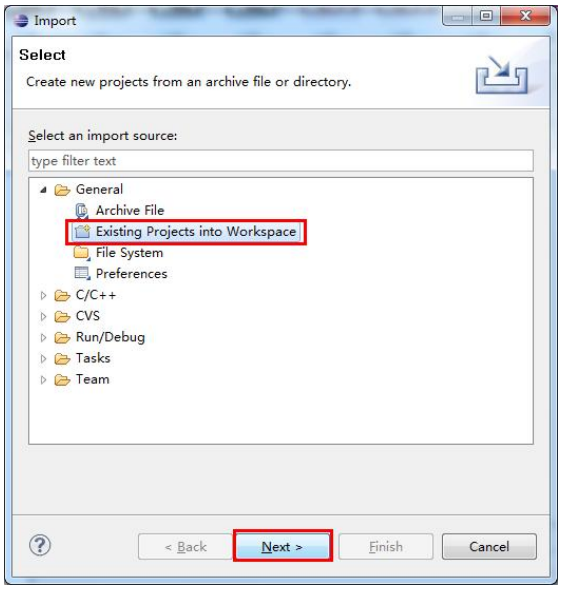
（注意：工程要放在英文路径下，不能有中文路径）

拷贝“华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\04-led”到 eclipse 工作目录下。如：D:\eclipse_projects\FS4412 目录。

如图所示，在 Project Explorer 窗口中右击鼠标，选择“Import...”，如下图所示：

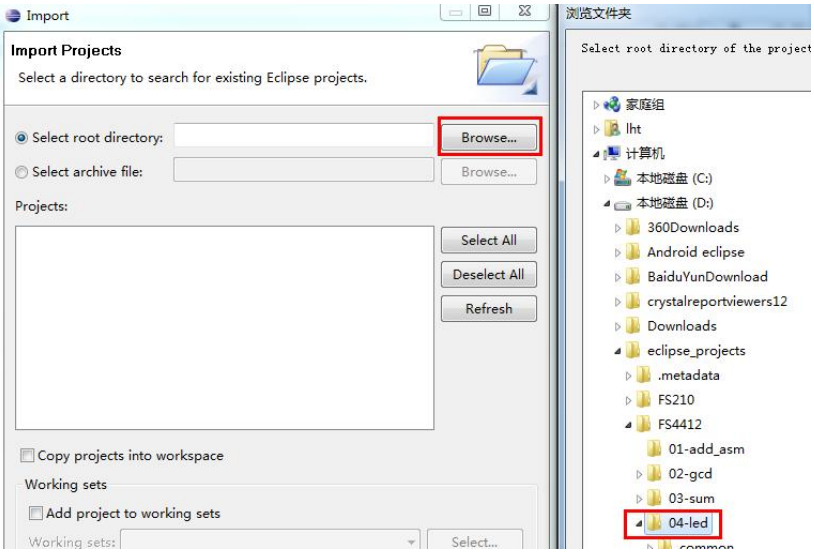


选择“Import...”后，出现如下图所示的窗口，选中“Existing Projects Into Workpace”然后点击“Next”。

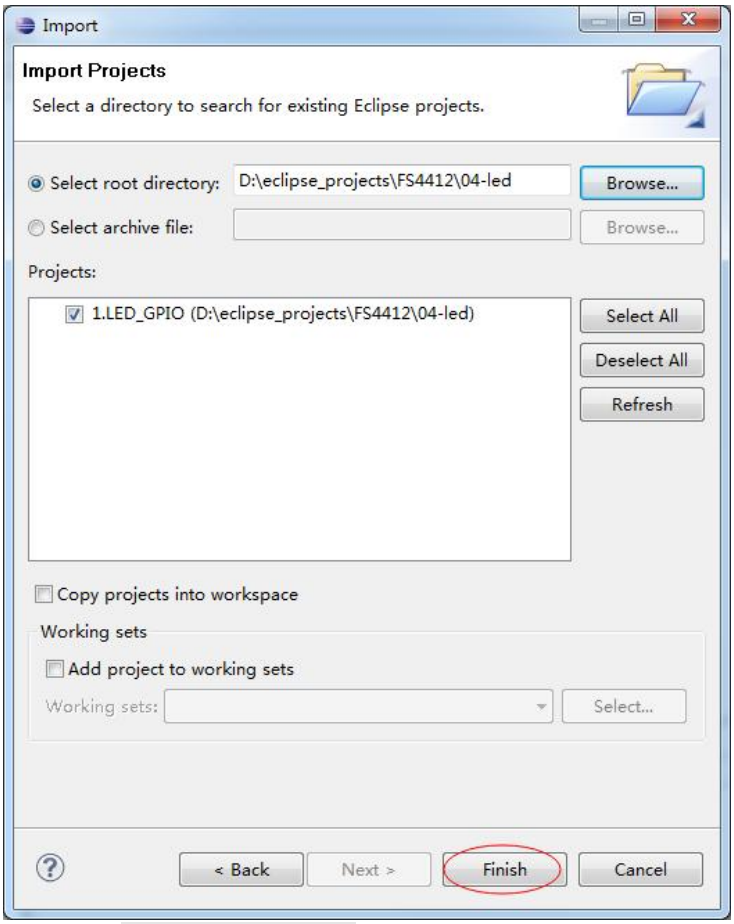


选择“Next”后出现如下窗口，点击“Browse...”→ 出现“浏览文件夹”窗口，

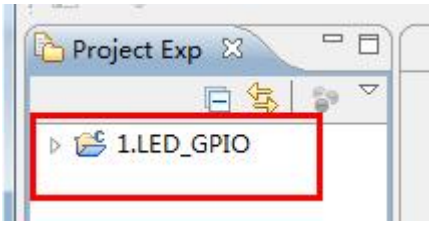
在“浏览文件夹”窗口中选中实验“04-led”后点击“确定”

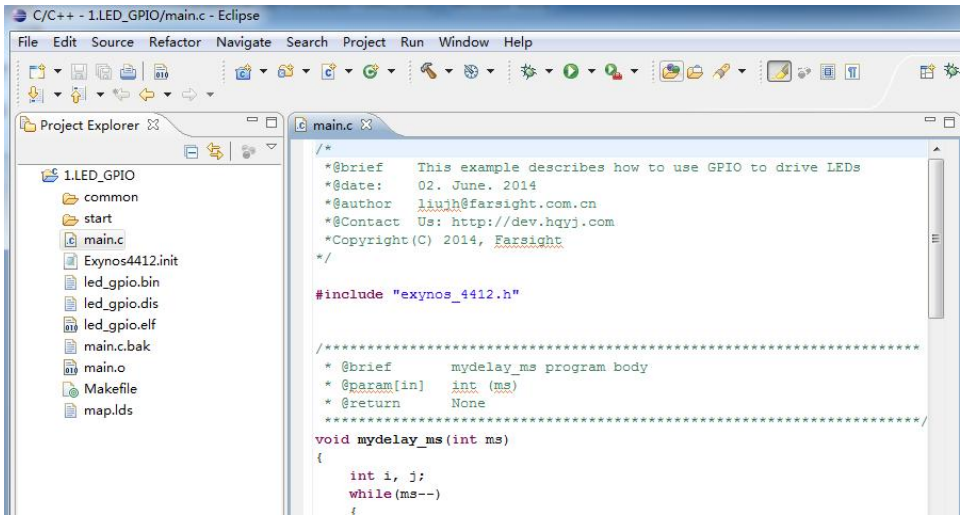


点击“确定”后出现如下所示窗口，直接点击“Finish”即可。



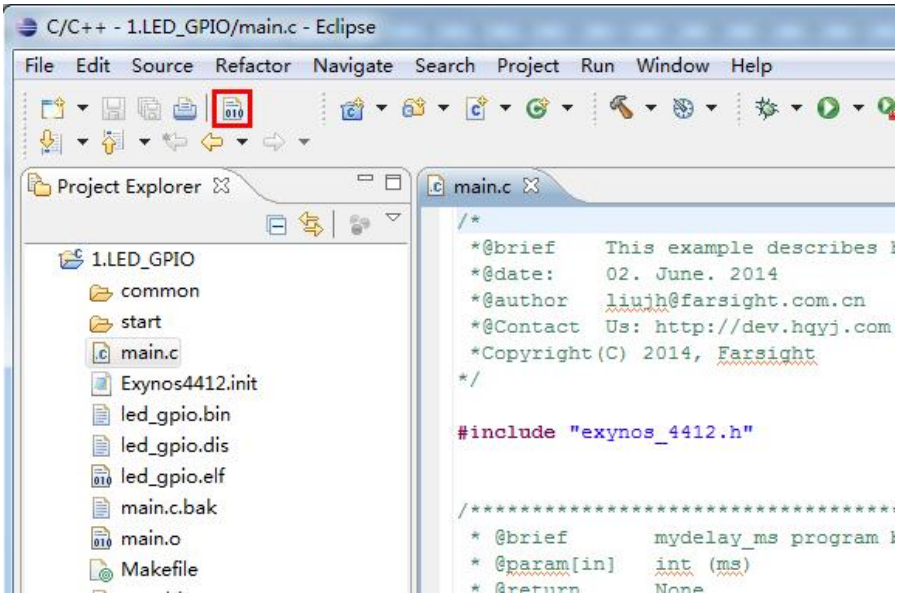
添加成功后可以在“Project Explorer”中看到“1.LED_GPIO”工程成功导入。



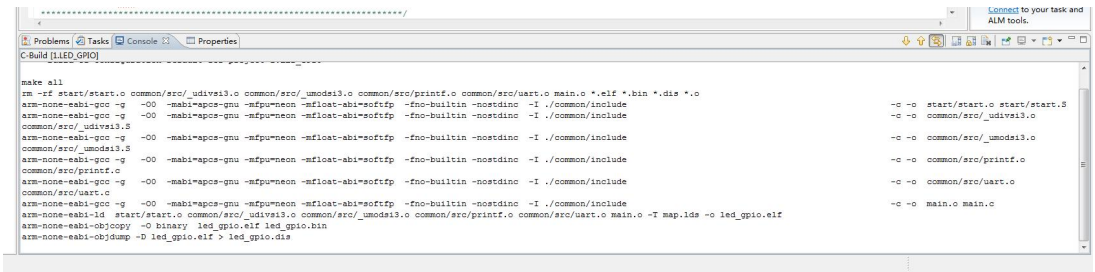


3. 编译程序

工程导入成功后，可以点击如下图所示的编译图标（或者快捷键“Ctrl + B”）



编译成功后如下图所示：



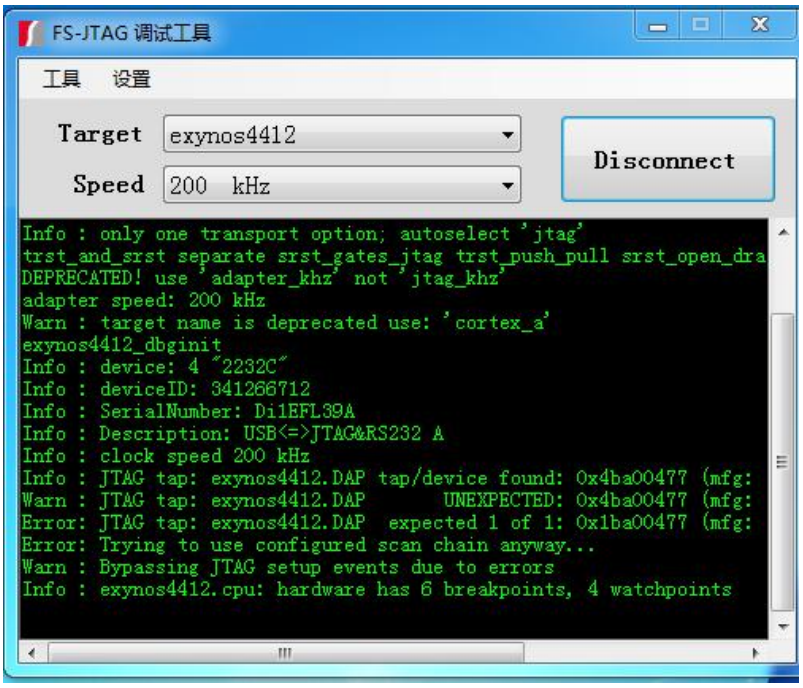
4. 调试工程

a. 配置 FS-JTAG 调试工具

打开 FS-JTAG 软件，在 Target 选项中选择 exynos4412，通信速率设置 200kHz。单击 Connect 按钮后，该按钮会变为 Disconnect，如图所示，即表示已经连接目标板。由于当前开发板还没有运行程序，仿真器无法获取相关信息，所以显示错误报告，是正常现象。

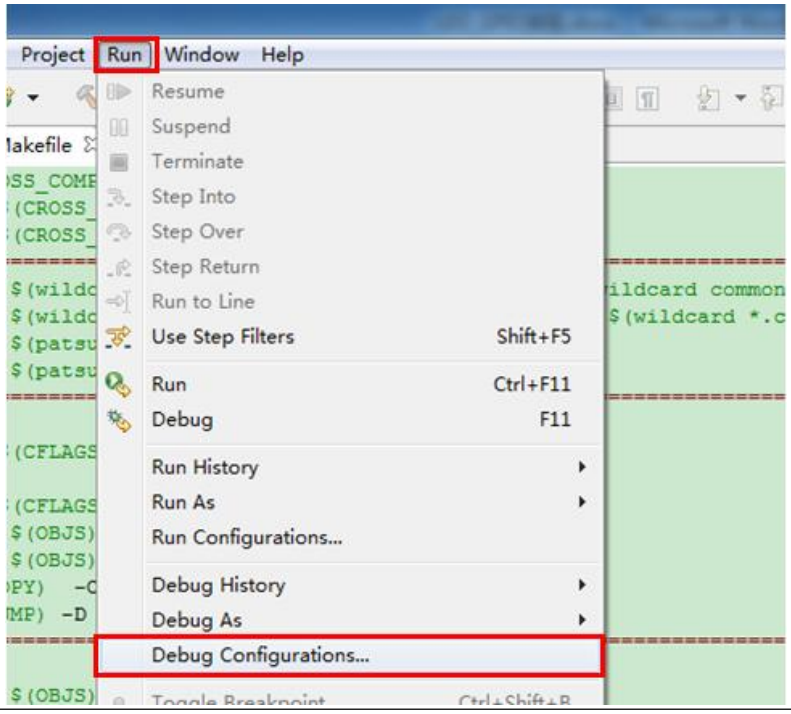
通信速率可以选择 200KHz，如果出现连接不稳定的情况，则将速率调制 100KHz。

注意：该软件在 win7 以上的 windows 操作系统中请使用管理员模式打开，否则会出现软件崩溃的现象。

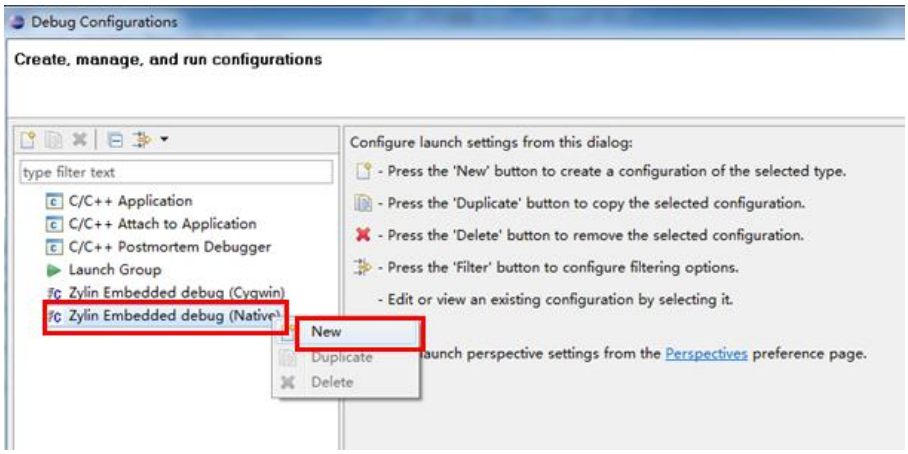


b. 配置调试工具

选中“Run”→“Debug Configuras…”

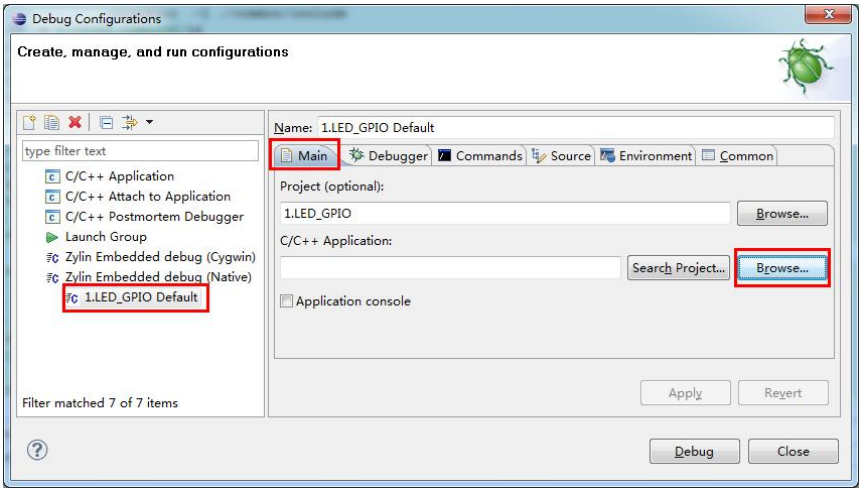


出现如下所示窗口，在“Zylin Embedded debug (Native)”选项上右击鼠标，在点击“New”选项

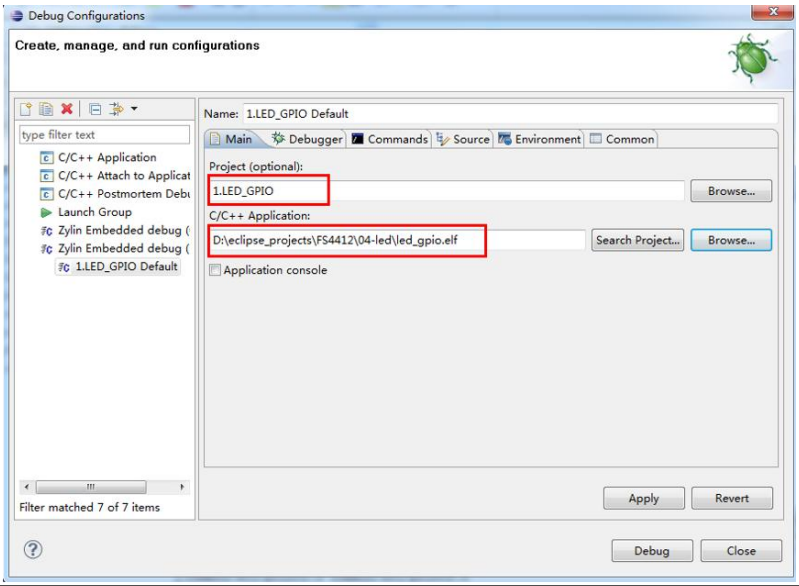


点击“New”选项后出现如下所示窗口

【main 选项卡】

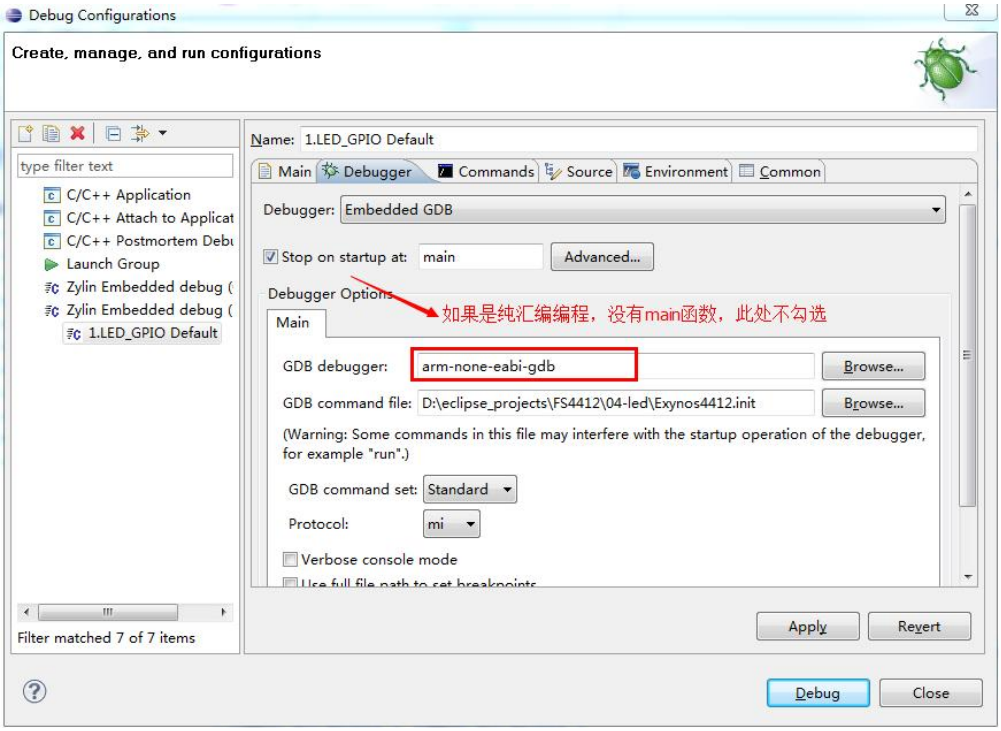


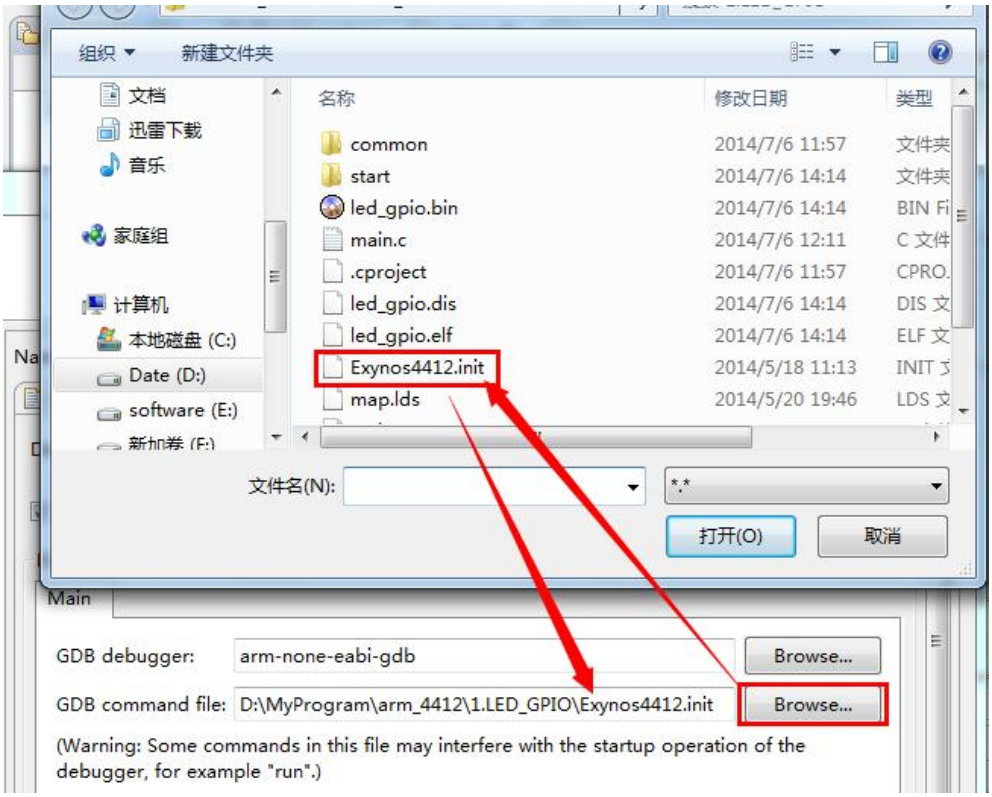
然后再点击“Main”选项栏中红色框内的“Browse”选项选择将要执行的程序，调试的程序格式为 xx**.elf** 的文件。找到 LED_GPIO 工程对应的目录，选中目录中的 **led_gpio.elf** 文件，然后选择“打开”，如下图所示。



【Debugger 选项卡】

在 Debugger 选项卡中的 Main 子选项卡中的 GDB debugger 的框中单击“Browse”按钮选择前面安装的 arm-none-eabi-gdb.exe（这里选择自己安装 gcc 编译工具的安装目录 C:\Program Files\yagarto\bin），在 GDB Command file 中选择自己工程目录下的 Exynos4412.init 文件，如图所示。



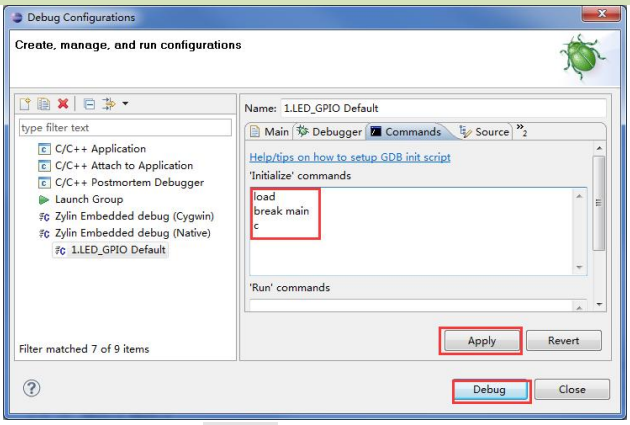


注：在 Debugger 选项卡中有一个 Stop on startup at：选项，如调试工程为 c 工程，有 main 函数则勾选该选项、否则不勾选。

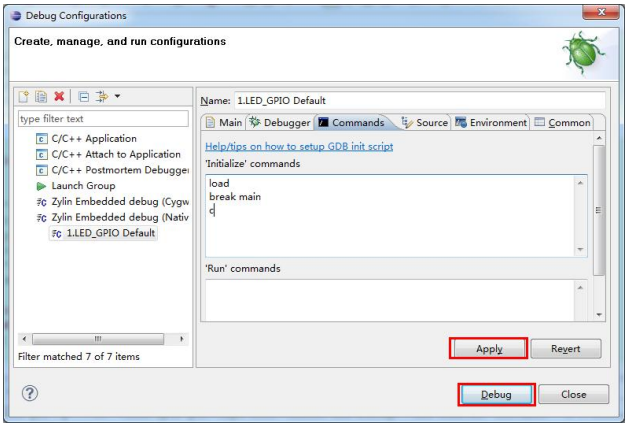
【Command 选项卡】

在 Command 选项中，如下图所示，在“Initialize’ commands”中添加 3 行命令

```
load
break main
c
```

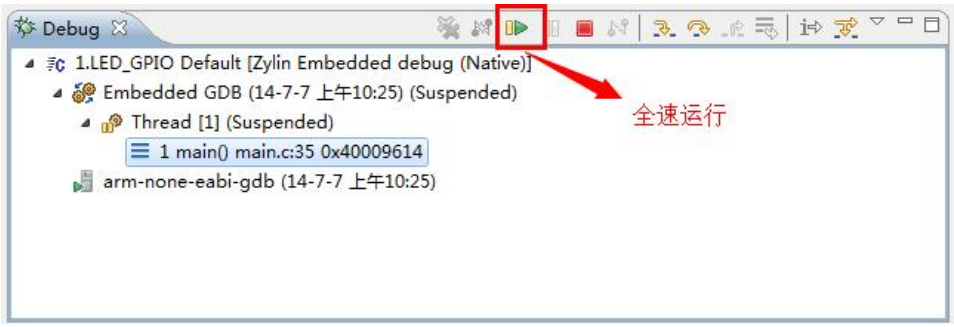


添加完后点击如下图所示的“Apply”，调试选项配置完成。再点击“Debug”选项进入就可以进行调试了。

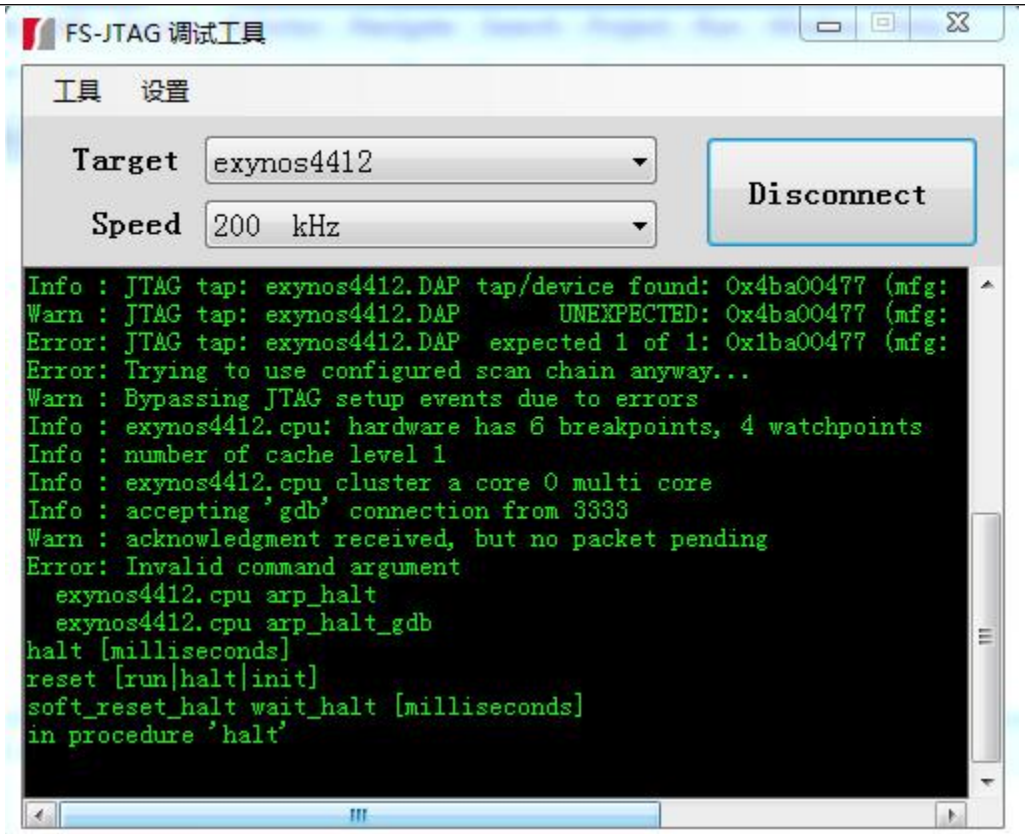


c. 进入调试

进入调试界面后，点击全速运行  的调试按钮，开发板上 **led 灯开始闪烁**，则表示整个开发环境搭建成功。

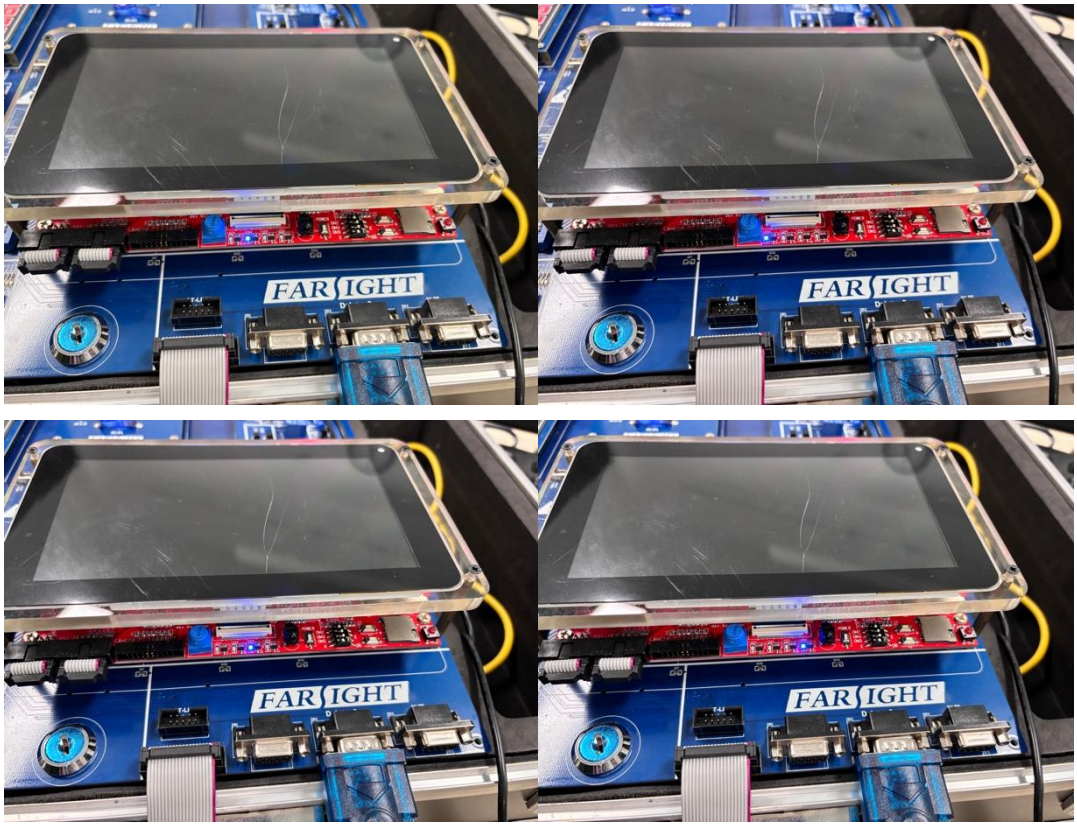


此时表示结束：



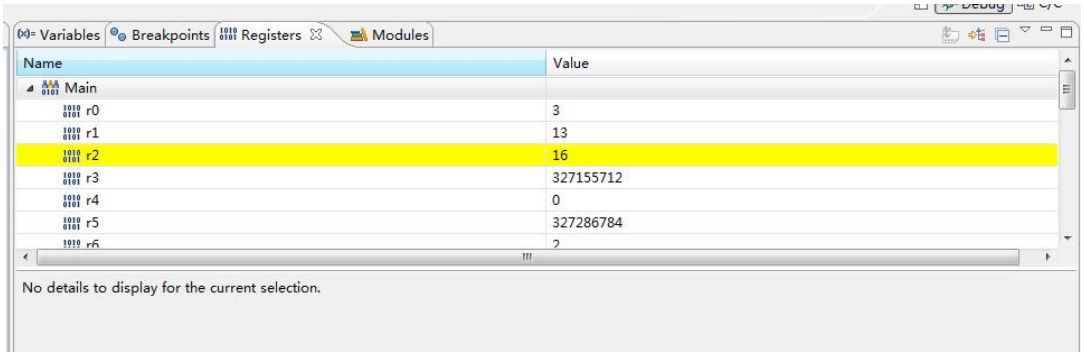
五、实验数据（现象）处理分析

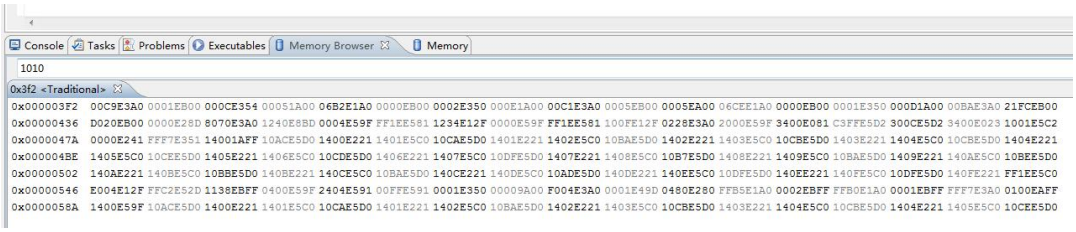
开发板上 **led 灯开始闪烁**，表明整个开发环境搭建成功。

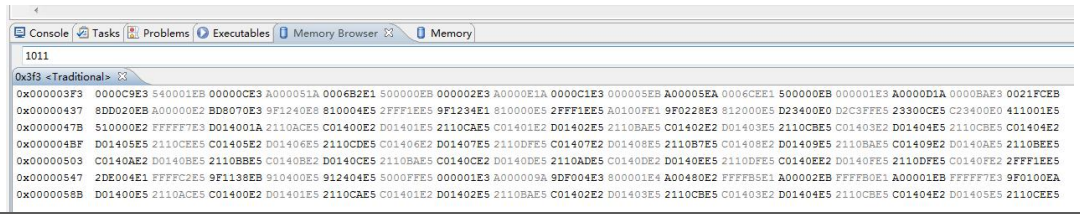


新建一个工程，编写一个汇编程序实现 3+13=16 的操作。

```
.globl _start
_start:
mov r0,#3
mov r1,#13
add r2,r1,r0
stop:
b stop
```







六、实验结论：

本次实验围绕 ARM 开发环境的搭建与应用展开，经过完整的实验流程，成功实现了环境配置、工程调试及功能验证，并通过额外的汇编编程实践加深了对 ARM 体系结构的理解。实验过程中，严格按照规范进行硬件连接、软件安装及代码测试，确保开发环境的稳定性和可用性，为后续的 ARM 开发奠定了坚实的基础。

1. ARM 开发环境的搭建

实验初期，通过安装必要的开发工具（如编译器、集成开发环境 IDE、调试工具）并正确配置相关驱动，实现了对 ARM 实验箱的识别与连接。实验过程中，确保了目标板与主机之间的通讯正常，保证了代码的下载与调试功能可用。成功的环境搭建为后续实验提供了良好的支持。

2. LED 工程导入与验证

在环境搭建完成后，实验通过导入预设的 LED 实验工程，进行代码编译、下载并运行。编译阶段，确保了代码无语法错误，并正确生成二进制可执行文件。随后，通过调试工具下载程序至 ARM 实验箱，使用全速运行模式进行测试，观察到 LED 灯正常闪烁，验证了开发环境的正确性，证明了代码的正确执行以及 ARM 处理器的基本运行能力。

3. 调试工具的使用与分析

实验过程中，熟悉并掌握了调试工具的基本操作，包括断点设置、单步执行、寄存器查看等功能。在调试过程中，通过设置断点并逐步执行代码，能够清晰地观察到程序的执行流程，并在寄存器窗口中查看相关寄存器的变化情况，加深了对 ARM 指令执行过程的理解。

4. 汇编语言编程与加法运算实验

为了进一步熟悉 ARM 的指令集架构，实验中额外使用汇编语言编写了一个简单的加法运算程序，实现 3 + 13 的计算。具体步骤如下：

[1]. 编写汇编代码：采用 ARM 指令完成加法运算，并将结果存储到指定寄存器中。

[2]. 程序下载与执行：使用调试工具将代码下载至 ARM 实验箱并运行。

[3]. 寄存器数据观察：通过调试界面查看通用寄存器的值，确认加法运算的正确性。

实验结果表明，ARM 处理器正确执行了加法指令，并在寄存器中存储了运算结果，进一步验证了开发环境的稳定性和 ARM 架构的基本运算能力。

5. 综合分析与总结

本次实验通过搭建 ARM 开发环境、运行 LED 实验工程以及汇编语言编程实验，成功验证了开发环境的稳定性，掌握了 ARM 指令的执行过程以及基本调试方法。LED 闪烁实验验证了硬件与软件的协同工作能力，而汇编加法实验加深了对 ARM 体系结构的理解，并培养了在嵌入式开发中进行底层调试的能力。

综合来看，本次实验不仅提升了对 ARM 嵌入式开发环境的掌握程度，同时也为后续深入学习 ARM 处理器的体系结构、指令集以及更复杂的嵌入式系统开发打下了坚实的基础。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。